

Transformer Architecture

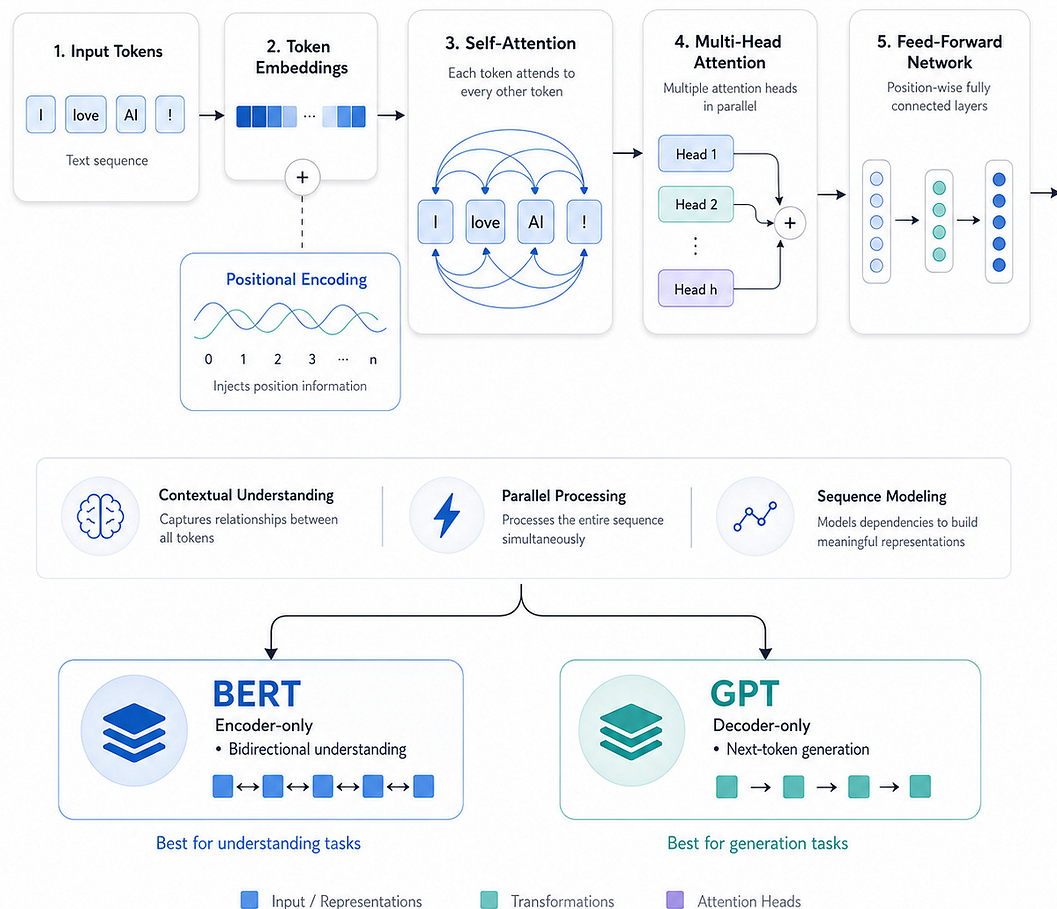
Self-Attention, Multi-Head Attention, Positional Encoding, BERT, GPT



AI ENGINEERING INSIDER

MAY 23, 2026

Transformer Architecture



A **Transformer** is a neural network architecture designed to process sequences using **attention instead of recurrence**.

The main idea:

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ **Subscribed**

- ↳ Every token can directly look at every other token.
- ↳ Attention decides which tokens matter most for understanding the current token.
- ↳ Multi-head attention lets the model learn different relationships in parallel.
- ↳ Positional encoding gives the model information about word order.
- ↳ BERT uses Transformers for bidirectional understanding.
- ↳ GPT uses Transformers for autoregressive next-token generation.

A strong interview answer should always connect the Transformer architecture to:

- ↳ Parallel training
- ↳ Long-range dependency modeling
- ↳ Contextual embeddings
- ↳ Attention-based reasoning
- ↳ Encoder vs decoder design
- ↳ Production tradeoffs: latency, memory, context length, inference cost

High-Level Transformer Architecture

A standard Transformer contains:

- ↳ Input token embeddings
- ↳ Positional encoding

- ↳ Self-attention layer
- ↳ Multi-head attention
- ↳ Feed-forward neural network
- ↳ Residual connections
- ↳ Layer normalization
- ↳ Output projection layer

For encoder models like **BERT**, the model reads the entire input bidirectionally.

For decoder models like **GPT**, the model predicts the next token using only previous tokens through causal masking.

Mock Interview Q&A

Q1. What problem did Transformers solve compared to RNNs and LSTMs?

Strong Answer:

Transformers solved two major limitations of RNNs and LSTMs: sequential processing and weak long-range dependency handling.

RNNs process tokens one at a time, which makes training slow and difficult to parallelize. Even with LSTMs, very long sequences can still suffer from information loss. Transformers replace recurrence with self-attention, allowing every token to attend directly to every other token in the sequence.

This makes Transformers more parallelizable, better at capturing long-range relationships, and more scalable for large language models.

Follow-up:

Why are Transformers easier to parallelize?

Answer:

Because all tokens in the input sequence can be processed at the same time during training. Unlike RNNs, where token 5 depends on token 4's hidden state, Transformer attention computes relationships across the entire sequence simultaneously using matrix operations.

Q2. Explain self-attention in simple terms.**Strong Answer:**

Self-attention is a mechanism that allows each token in a sequence to decide how much attention it should pay to every other token.

For example, in the sentence:

"The animal did not cross the street because it was tired."

The word "it" is likely to refer to "animal." Self-attention helps the model assign a higher weight to "animal" when processing "it."

Technically, each token is transformed into three vectors:

↳ Query

↳ Key

↳ Value

The query asks: "What am I looking for?"

The key says: "What information do I contain?"

The value says: "What information should I pass forward?"

Attention scores are computed between queries and keys, then used to weight the values.

Q3. What is the formula for scaled dot-product attention?

Strong Answer:

The formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where:

↳ Q = query matrix

↳ K = key matrix

↳ V = value matrix

↳ d_k = dimension of the key vectors

↳ $\sqrt{d_k}$ = scaling factor to prevent large dot products

The softmax converts similarity scores into probabilities, which are then used to compute a weighted sum of the value vectors.

Q4. Why do we divide by the square root of d_k in attention?

Strong Answer:

We divide by $\sqrt{d_k}$ to stabilize the attention scores.

When the key and query vectors have high dimensionality, their dot products can become large. Large values passed into softmax can make the distribution extremely sharp, meaning one token gets almost all the attention while others get near zero.

This causes poor gradient flow and unstable training. Scaling by $\sqrt{d_k}$ keeps the variance of the dot products controlled and makes the softmax smoother.

Q5. What is multi-head attention?

Strong Answer:

Multi-head attention means running several attention mechanisms in parallel.

Instead of having a single attention head learn all relationships, the model splits the representation across multiple heads. Each head can specialize in different patterns.

For example:

- ↳ One head may focus on subject-verb relationships.
- ↳ Another may focus on positional dependency.
- ↳ Another may capture coreference.
- ↳ Another may capture semantic similarity.

After all heads compute attention, their outputs are concatenated and projected back into the model dimension.

Q6. Why is multi-head attention better than single-head attention?

Strong Answer:

Single-head attention gives the model only one representation of token relationships. Multi-head attention allows the model to learn multiple relationship types simultaneously.

This improves expressiveness because language has many overlapping structures: syntax, semantics, position, entity relationships, and discourse dependencies.

In production LLMs, multi-head attention also enables different parts of the model to specialize, thereby improving representation quality and downstream performance.

Q7. What is positional encoding, and why is it needed?

Strong Answer:

Transformers do not process tokens sequentially like RNNs. Because self-attention treats input tokens as a set, the model has no built-in understanding of word order.

Positional encoding injects position information into the token embeddings so the model knows whether a token appears first, second, third, and so on.

Without positional encoding, the sentences:

“Dog bites man.”

and

“Man bites dog.”

could appear too similar to the model because the tokens are the same, but the order changes the meaning completely.

Q8. What are the main types of positional encoding?

Strong Answer:

There are several types:

↳ **Sinusoidal positional encoding**

Uses fixed sine and cosine functions. It allows the model to generalize to longer positions than seen during training.

↳ **Learned positional embeddings**

The model learns position vectors during training. This is common in models like BERT and GPT-style architectures.

↳ **Rotary positional embeddings, or RoPE**

Injects position by rotating query and key vectors. It is common in modern LLMs because it handles relative position relationships well.

↳ **ALiBi**

Adds linear position bias to attention scores, helping with extrapolation to longer contexts.

Q9. What is the difference between encoder, decoder, and encoder-decoder Transformers?

Strong Answer:

An **encoder** reads the full input sequence and builds contextual representations. BERT is an encoder-only model.

A **decoder** generates output tokens one at a time using causal masking. GPT is a decoder-only model.

An **encoder-decoder** model uses an encoder to understand the input and a decoder to generate the output. This is common in translation and summarization models, such as the original Transformer and T5.

The key difference is masking and objective:

- ↳ Encoder: bidirectional understanding
 - ↳ Decoder: left-to-right generation
 - ↳ Encoder-decoder: input understanding plus output generation
-

Q10. How does BERT use the Transformer architecture?

Strong Answer:

BERT is an encoder-only Transformer model. It uses bidirectional self-attention, meaning each token can attend to tokens on both its left and right sides.

BERT was trained mainly using masked language modeling. Some tokens are masked, and the model learns to predict them based on the surrounding context.

Because BERT sees both previous and future tokens, it is strong for understanding tasks such as:

- ↳ Text classification
- ↳ Named entity recognition
- ↳ Sentiment analysis
- ↳ Question answering
- ↳ Semantic search embeddings

BERT is not naturally designed for open-ended text generation because it is not autoregressive.

Q11. How does GPT use the Transformer architecture?

Strong Answer:

GPT is a decoder-only Transformer model. It uses causal self-attention, meaning each token can only attend to previous tokens, not future tokens.

GPT is trained using next-token prediction. Given previous tokens, it learns to predict the next token.

This makes GPT strong for generative tasks such as:

- ↳ Chatbots
- ↳ Text generation
- ↳ Code generation
- ↳ Summarization
- ↳ Reasoning-style completion
- ↳ Tool-calling agents

The causal mask is essential because it prevents the model from cheating by looking at future tokens during training.

Q12. What is causal masking?

Strong Answer:

Causal masking prevents a token from attending to future tokens.

In GPT-style models, when predicting token t , the model should only see tokens from positions 1 to $t-1$.

For example, in the sequence:

“I love machine learning.”

When predicting “machine,” the model can attend to “I” and “love,” but not “learning.”

This preserves the autoregressive generation objective and makes the model suitable for next-token prediction.

Q13. What is the difference between BERT and GPT?

Strong Answer:

BERT and GPT are both Transformer-based, but they are optimized for different tasks.

BERT is encoder-only and bidirectional. It is better for understanding tasks where the full input is available.

GPT is decoder-only and autoregressive. It is better for generation tasks where the model produces text one token at a time.

BERT learns through masked language modeling, while GPT learns through next-token prediction.

A simple way to explain it:

- ↳ BERT is better at understanding text.
 - ↳ GPT is better at generating text.
 - ↳ BERT looks both left and right.
 - ↳ GPT only looks left during generation.
-

Q14. Why are residual connections used in Transformers?

Strong Answer:

Residual connections help gradients flow through deep networks.

In Transformers, each attention and feed-forward block is wrapped with a residual connection. Instead of forcing each layer to completely transform the input, the model learns a refinement on top of the previous representation.

This improves training stability, helps avoid vanishing gradients, and allows very deep Transformer models to train effectively.

Q15. What is layer normalization doing in Transformers?

Strong Answer:

Layer normalization stabilizes the hidden representations inside the Transformer.

It normalizes activations across the feature dimension for each token. This helps maintain a stable distribution of activations as data flows through many layers.

In modern Transformers, there are two common patterns:

↳ **Post-LN:** layer normalization after the residual block

↳ **Pre-LN:** layer normalization before the attention or feed-forward block

Pre-LN is often preferred in large models because it improves training stability.

Q16. What is the feed-forward network in a Transformer block?

Strong Answer:

After attention, each token representation passes through a position-wise feed-forward network.

This is usually a two-layer MLP applied independently to each token.

The attention layer mixes information across tokens, while the feed-forward layer transforms each token's representation internally.

A common structure is:

- ↳ Linear projection up
- ↳ Nonlinear activation
- ↳ Linear projection down

Modern LLMs often use activations like GELU, SwiGLU, or GeLU-based variants.

Q17. What is the computational bottleneck of Transformers?

Strong Answer:

The main bottleneck is self-attention complexity.

For a sequence length n , standard self-attention has approximately quadratic complexity because every token attends to every other token.

That means memory and compute scale roughly with n^2 .

This becomes expensive for long-context models, document processing, video understanding, code repositories, and agent memory systems.

Common optimization strategies include:

- ↳ Sparse attention
 - ↳ Sliding-window attention
 - ↳ FlashAttention
 - ↳ KV caching
 - ↳ Retrieval-augmented generation
 - ↳ Chunking and re-ranking
 - ↳ Long-context architectures
-

Q18. What is KV cache in GPT inference?

Strong Answer:

KV cache stores the key and value tensors from previous tokens during autoregressive generation.

Without KV cache, the model would recompute attention over the entire previous context every time it generates a new token.

With KV cache, the model reuses previous keys and values and only computes the new token's query, key, and value.

This significantly improves decoding speed, especially for long outputs.

The trade-off is memory usage, as the cache grows with sequence length, the number of layers, the number of heads, and the hidden size.

Q19. Why does GPT generate one token at a time?

Strong Answer:

GPT is autoregressive. Each generated token becomes part of the context for the next prediction.

For example:

Input: "The capital of France is"

The model predicts: "Paris."

Then the next input becomes:

"The capital of France is Paris."

This sequential decoding is why GPT inference is harder to parallelize than training. Training can process full sequences in parallel with masking, but generation depends on previously generated tokens.

Q20. How would you explain attention to a non-technical stakeholder?

Strong Answer:

I would explain attention as a relevance scoring mechanism.

When the model reads a sentence, it does not treat every word equally. It decides which words are most relevant to understanding the current word.

For example, in customer support, if the user says:

"My order arrived late and the package was damaged."

The model should pay strong attention to "order," "late," and "damaged" because those words determine the user's intent and the next action.

Attention helps the model focus on the most useful parts of the input.

Senior-Level Follow-Up Questions

Q21. How would you optimize a Transformer model for production inference?

Strong Answer:

I would optimize at multiple layers:

↳ **Model-level optimization**

Use quantization, pruning, distillation, or smaller task-specific models.

↳ **Serving optimization**

Use batching, paged attention, KV cache, streaming output, and efficient runtimes like TensorRT-LLM or vLLM.

↳ **Prompt optimization**

Reduce unnecessary context, compress instructions, and use retrieval only when needed.

↳ **System optimization**

Cache embeddings, cache retrieval results, cache frequent completions, and monitor token usage.

↳ **Quality control**

Track latency, cost, hallucination rate, grounding score, and user satisfaction.

A strong production answer should not only say “use a smaller model.” It should connect latency, cost, memory, throughput, and quality tradeoffs.

Q22. How does attention relate to hallucination?

Strong Answer:

Attention itself does not guarantee factuality. It only learns token relationships based on training data and context.

A model may attend strongly to misleading, incomplete, or irrelevant context. It may also generate plausible text from learned patterns without grounding in verified knowledge.

To reduce hallucination, I would combine Transformer-based generation with:

- ↳ Retrieval-augmented generation
- ↳ Source-grounded prompting
- ↳ Citation validation
- ↳ Tool use
- ↳ Confidence scoring
- ↳ Human escalation
- ↳ Post-generation verification

In production systems, hallucination is not solved only at the model architecture level. It must be handled at the system design level.

Q23. How would you test a Transformer-based AI system?

Strong Answer:

I would test it across several layers:

- ↳ **Unit tests** for tokenization, preprocessing, embeddings, retrieval, and prompt formatting
- ↳ **Model evaluation** for accuracy, F1, BLEU, ROUGE, BERTScore, or task-specific metrics
- ↳ **RAG evaluation** for context precision, context recall, faithfulness, and answer relevance
- ↳ **Safety testing** for jailbreaks, prompt injection, toxicity, and privacy

leakage

- ↳ **Latency testing** for p50, p95, and p99 response time
- ↳ **Cost testing** for token usage and model call frequency
- ↳ **Regression testing** to detect quality drops after prompt, model, or data changes
- ↳ **Online monitoring** for drift, hallucination, user feedback, and failure cases

For AI Engineering roles, I would emphasize that testing is not only about model accuracy. It includes reliability, safety, cost, and production observability.

Q24. How would you explain why GPT needs large-scale training data?

Strong Answer:

GPT learns language patterns through next-token prediction. To become useful across many domains, it needs exposure to diverse examples of grammar, facts, reasoning patterns, code, conversations, and instructions.

Large-scale data helps the model learn general representations. However, more data is not always better. Data quality, deduplication, filtering, domain balance, and safety alignment are also critical.

For production use, I would also distinguish between:

- ↳ Pretraining for general capability
 - ↳ Fine-tuning for behavior specialization
 - ↳ RAG for external knowledge grounding
 - ↳ Prompting for task-specific control
-

Q25. When would you use BERT instead of GPT?

Strong Answer:

I would use BERT when the task is primarily understanding, classification, ranking, or embedding-based retrieval.

Examples:

- ↳ Sentiment classification
- ↳ Named entity recognition
- ↳ Search relevance
- ↳ Document similarity
- ↳ Intent classification
- ↳ Extractive question answering

I would use GPT when the task requires generation, reasoning, conversation, or dynamic response creation.

Examples:

- ↳ Chat assistant
- ↳ Report generation
- ↳ Code generation
- ↳ Summarization
- ↳ Agentic workflows
- ↳ Tool-calling

The decision depends on task objective, latency, cost, and whether generation is required.

Tough Interview Questions

Q26. Why does attention have quadratic complexity?

Strong Answer:

Attention compares every token with every other token.

If the sequence has n tokens, each token computes attention scores against n tokens. That creates an $n \times n$ attention matrix.

So as sequence length grows, memory and compute grow quadratically.

This is why long-context inference becomes expensive and why many production systems use chunking, retrieval, sparse attention, or optimized kernels.

Q27. Why does Transformer training parallelize better than inference?**Strong Answer:**

During training, the model already has the full target sequence. With causal masking, it can compute predictions for all positions in parallel.

During inference, GPT does not know future tokens yet. It must generate one token, append it to the context, then generate the next token.

So training is parallelizable across sequence positions, but autoregressive inference is sequential across generated tokens.

Q28. What happens if positional encoding is removed?**Strong Answer:**

The Transformer would lose explicit information about token order.

Self-attention would still know token content, but it would not know whether a word appeared early or late in the sequence.

This can break the meaning in language because word order matters. For example:

“User deleted admin.”

and

“Admin deleted user.”

contain similar tokens but very different meanings.

Without position information, the model would struggle to distinguish these structures.

Q29. What is the difference between attention weights and explanation?

Strong Answer:

Attention weights show which tokens influenced a representation mathematically, but they are not always a faithful human explanation.

A token receiving close attention does not always mean it is the causal reason for the final prediction. Other layers, heads, residual paths, and feed-forward transformations also affect the output.

So I would treat attention maps as a diagnostic signal, not a complete explanation.

For production interpretability, I would combine attention inspection with attribution methods, counterfactual tests, retrieval traces, and output validation.

Q30. How would you answer: “Design a Transformer-based customer support assistant”?

Strong Answer:

I would design it as a production-grade LLM system, not just a raw Transformer model.

The architecture would include:

- ↳ User query input
- ↳ Intent classification
- ↳ Retrieval from product docs, FAQ, policies, and ticket history
- ↳ Re-ranking for the best context
- ↳ GPT-style generation with grounded context
- ↳ Tool calling for order status, refunds, appointments, or account actions
- ↳ Guardrails for policy, privacy, and unsafe requests
- ↳ Verification layer to check whether the response is supported by the retrieved evidence
- ↳ Escalation to human support when confidence is low
- ↳ Observability for traces, latency, token cost, hallucination rate, and user feedback

The Transformer provides the reasoning and language generation layer, but the full AI engineering solution requires retrieval, tools, evaluation, monitoring, and governance.

Best Short Interview Answer

Use this when the interviewer asks:

“Explain Transformer architecture.”

Answer:

A Transformer is a neural architecture for sequence modeling that uses self-attention instead of recurrence. Each token is converted into query, key, and value vectors. Attention scores determine how much each token should focus on other tokens. Multi-head attention allows the model to learn different relationships in parallel, such as syntax, semantics, and long-range dependencies.

Because Transformers do not process tokens sequentially like RNNs, they need positional encoding to represent word order. A Transformer block usually contains multi-head attention, feed-forward layers, residual connections, and layer normalization.

BERT uses an encoder-only Transformer with bidirectional attention, making it strong for understanding tasks. GPT uses a decoder-only Transformer with causal masking, making it strong for next-token generation and conversational AI.

In production, the main tradeoffs are memory, latency, context length, inference cost, and hallucination control.

Common Mistakes to Avoid

- ↳ Saying Transformers understand order naturally. They do not; they need positional information.
- ↳ Saying attention is the same as explanation. Attention can help inspect behavior, but it is not a complete explanation.
- ↳ Confusing BERT and GPT. BERT is a bidirectional encoder-only; GPT is an autoregressive decoder-only.
- ↳ Forgetting causal masking in GPT.

↳ Ignoring inference cost. Transformer attention becomes expensive as context length grows.

↳ Talking only about model architecture and not production concerns.

Final Interview Cheat Sheet

Remember this structure:

Transformer = Embeddings + Position + Attention + Feed Forward + Residual + Normalization

Self-attention:

↳ Each token attends to other tokens.

Multi-head attention:

↳ Multiple attention patterns are learned in parallel.

Positional encoding:

↳ Adds word order information.

BERT:

↳ Encoder-only, bidirectional, strong for understanding.

GPT:

↳ Decoder-only, causal, strong for generation.

Main bottleneck:

↳ Attention scales quadratically with sequence length.

Production concerns:

↳ Latency, cost, memory, hallucination, context management, evaluation, and monitoring.

This Substack is reader-supported. To

receive new posts and support my work,
consider becoming a free or paid subscriber.

✓ **Subscribed**